

Problema de Rutas — BFS y UCS (v1)

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Proyecto	2
2. Estructura del Grafo	2
3. BFS — Búsqueda por Amplitud	2
4. UCS — Búsqueda de Costo Uniforme	3
5. Diferencia entre BFS y UCS	3
6. Reconstrucción del Camino	3
7. Interfaz Gráfica	4

1. Descripción del Proyecto

Visualizador de planificación de rutas sobre un grafo de ciudades mexicanas usando **BFS** (Búsqueda por Amplitud) y **UCS** (Búsqueda de Costo Uniforme). El usuario selecciona ciudad origen y destino, y el programa encuentra y muestra el camino óptimo.

2. Estructura del Grafo

El grafo se representa como una **lista de adyacencia**. Cada ciudad tiene una lista de vecinos con sus distancias en kilómetros:

```
1 Map<String, List<Arista>> grafo = new HashMap<>();
2
3 grafo.put("Acapulco", Arrays.asList(
4     new Arista("Ciudad de México", 400),
5     new Arista("Puebla", 480)
6 ));
7 // ... m s ciudades
```

Las ciudades incluyen: Acapulco, Ciudad de México, Puebla, Veracruz, Oaxaca, Poza Rica, Huatulco, Querétaro, Pachuca, entre otras.

3. BFS — Búsqueda por Amplitud

BFS explora todos los vecinos de un nodo antes de avanzar al siguiente nivel. Garantiza el camino con el **mínimo número de saltos** (ciudades intermedias).

```
1 Nodo_BFS resolverBFS(String origen, String destino) {
2     Queue<Nodo_BFS> cola = new LinkedList<>();
3     Set<String> visitados = new HashSet<>();
4     cola.add(new Nodo_BFS(origen, 0, null));
5     visitados.add(origen);
6
7     while (!cola.isEmpty()) {
8         Nodo_BFS actual = cola.poll();
9         if (actual.ciudad.equals(destino)) return actual;
10
11         for (Arista a : grafo.get(actual.ciudad)) {
12             if (!visitados.contains(a.destino)) {
13                 visitados.add(a.destino);
14                 cola.add(new Nodo_BFS(a.destino,
15                     actual.costo + a.peso, actual));
16             }
17         }
18     }
19     return null;
20 }
```

4. UCS — Búsqueda de Costo Uniforme

UCS usa una **cola de prioridad** ordenada por costo acumulado. Siempre expande el nodo de menor costo total, garantizando la ruta de **menor distancia**.

```
1  Nodo_BFS resolverUCS(String origen, String destino) {
2      PriorityQueue<Nodo_BFS> cola =
3          new PriorityQueue<>(
4              Comparator.comparingInt(n -> n.costo));
5      Set<String> visitados = new HashSet<>();
6      cola.add(new Nodo_BFS(origen, 0, null));
7
8      while (!cola.isEmpty()) {
9          Nodo_BFS actual = cola.poll();
10         if (visitados.contains(actual.ciudad)) continue;
11         visitados.add(actual.ciudad);
12
13         if (actual.ciudad.equals(destino)) return actual;
14
15         for (Arista a : grafo.get(actual.ciudad)) {
16             if (!visitados.contains(a.destino)) {
17                 cola.add(new Nodo_BFS(a.destino,
18                     actual.costo + a.peso, actual));
19             }
20         }
21     }
22     return null;
23 }
```

5. Diferencia entre BFS y UCS

Característica	BFS	UCS
Estructura	Cola FIFO	Cola de prioridad
Criterio de expansión	Nivel (saltos)	Costo acumulado
Solución óptima	En saltos	En distancia total
Considera pesos	No	Sí

6. Reconstrucción del Camino

Cada nodo guarda referencia al nodo padre. La ruta se reconstruye recursivamente:

```
1  List<String> reconstruirRuta(Nodo_BFS nodo) {
2      List<String> ruta = new ArrayList<>();
3      while (nodo != null) {
4          ruta.add(0, nodo.ciudad);
5          nodo = nodo.padre;
6      }
7      return ruta;
8  }
```

7. Interfaz Gráfica

- Imagen del grafo de ciudades como fondo visual.
- Combos para seleccionar ciudad origen y destino.
- Botones **BFS** y **UCS** para ejecutar cada algoritmo.
- Resultado mostrado: ruta completa, distancia total y número de ciudades.